AI Agents for Economic Research & Teaching

AI Exchange

Lucas Bütje

2026

University of Konstanz

It all starts with a folder.

Where we are going for the next 90 minutes

1. What is an AI agent?
Chatbot vs. agent, capabilities, risks

2. How to use one well
Context, memory, planning, skills, teams, tools, hooks, verification

3. The reveal
We open what it built, and inspect the receipts

4. Exchange
Your experiences: what worked, what failed

Examples throughout come from real research and teaching.

What is an AI agent?

You open an agent inside a folder: that folder is its world

You launch the agent *in* a project folder. Everything happens there.

```
my-project/          <- you start the agent here
  CLAUDE.md            project context (the rules)
  log/current.md       memory across sessions
  data/  code/  ...    your files
```

- ✔ Reads, writes, acts **inside this folder**
- ⚠ Cannot reach files **above** it

A collaborator that lives *inside* one project, not a chatbot in the cloud.

We all use LLM chatbots. Agents can do more

Chatbots

A chat window.
You type, the AI replies with text.

Claude ▪ ChatGPT ▪ Gemini ▪ Mistral

- ✗ No access to your computer
- ✗ No files, no commands
- ✗ Text in, text out

Agentic AI

A program on your computer.
The AI acts on its own.

Claude Code ▪ Codex ▪ Antigravity ▪ OpenCode

- ✓ Reads and writes files
- ✓ Runs terminal commands
- ✓ Completes multi-step tasks

An agent can do anything your computer can

- **Read and write files:** text, code, PDF, spreadsheets
- **Run code:** Python, R, Bash, compile LaTeX
- **Terminal commands:** install packages, Git, any system command
- **Search the web:** read documentation, check sources
- **Multi-step tasks:** plan, execute, fix errors, keep going

In principle, the AI can do anything on your computer that an IT expert could.

That power brings real risks

Risks

- File access is **real**: files can be overwritten or deleted
- Plugins (MCPs) from unknown sources can inject malicious code
- Sensitive data in the working folder is readable by the AI

Workflow implications

- **Plan mode** first: review changes before they run
- **Back up** important files regularly
- **Install** only what you **trust**
- **No sensitive data** in the working folder

The tools guard you, but never completely

Agents like **Claude Code**, **Codex**, **Antigravity** and **OpenCode** build in safeguards:

- ✔ **Permission before each action:** before a file is written or a command runs, it asks; you see exactly what is about to happen
- ✔ **Plan / read-only mode:** the AI thinks first, you approve
- ✔ **Confined to the working folder:** no access above the opened project folder
- ✔ **Changes are visible:** file diffs are shown before they are applied

Stay alert anyway:

- ⚠ LLMs use **randomness**: same prompt, different output; sometimes a wrong one
- ⚠ The tools are **not perfect** and keep changing. Safeguards can fail.
- ⚠ Permission pop-ups are quickly **clicked away**. Do not confirm blindly.

How to use an agent well (in my experience)

None of the following requires manual setup on your part.

Just ask the agent:

"Set up a `CLAUDE.md`"
"Create a skill based on this github link, adapted for my workflow."
"Use one subagent per paper."

An agent is a brilliant RA — with no context and no memory

Think of it as a **highly capable, fast, tireless research assistant**.

Without context

- ⚠ It doesn't know your project, your conventions, your data
- ⚠ So it guesses (→ errors) or works it out the slow way (→ wasted time)

Without memory

- ⚠ It forgets everything between sessions
- ⚠ Every new session is a **fresh** RA, starting again from day one

So give it what any good RA needs: **context** and **memory**.

1. `CLAUDE.md` gives the agent your context, once

A plain Markdown file the agent reads at the *start of every session*. Two levels, and they stack:

Global `~/.claude/CLAUDE.md`

Applies *everywhere*: **how you work**.

- Never overwrite raw data
- Don't make research design decisions without me
- German writing conventions

Local `<project>/CLAUDE.md`

Applies *here* only: **what this project is**.

- Data lives in `data/wave6/`
- This deck: build with the `deck` skill
- Codebook in `docs/codebook.md`

Why it matters: write it once and the agent follows it forever. You stop re-explaining how you work, and how *this* project is laid out, at the top of every chat.

2. A running log gives the agent memory across sessions

Models forget everything between sessions. A habit I adopted from **Scott Cunningham**: keep one working-memory file, `log/current.md`, and let the agent maintain it.

The loop:

1. **As agent works:** the agent appends short entries: what changed, why, what is still open
2. **At the next start:** the file is read back in automatically, so it resumes where you stopped
3. Say **"log"**: archive `current.md` to a timestamped file, start a fresh one

```
2026-06-24 -- Cleaned wave 6, merged on pidp. Open: 142
  duplicate household ids -- decide merge key next time.
```

3. Plan mode lets you approve before anything happens

The agent investigates and writes a **plan**, but changes *nothing* until you say go. The single most useful safety habit.

Without plan mode

- ⚠ It starts editing immediately
- ⚠ You discover a wrong assumption *after* 12 files changed

With plan mode

- ✓ You read the plan, fix the misunderstanding
- ✓ Only then does it touch your files

>_ **Live demo:** Let's try creating a set of exercises to practice what this talk covers

4. Skills package expertise you reuse

A skill is a reusable, named recipe: expert instructions you (or others) wrote once, invoked with one command.

Research

- `/summarize-paper`: structured summary
- `/referee2`: audit a complete code pipeline
- `/blindspot`: find what you can't see in your own results

Teaching & craft

- `/quiz-review`: check a quiz for correctness
- `/deck`: build a deck (this one!)
- `/germancheck`: catch translation tells

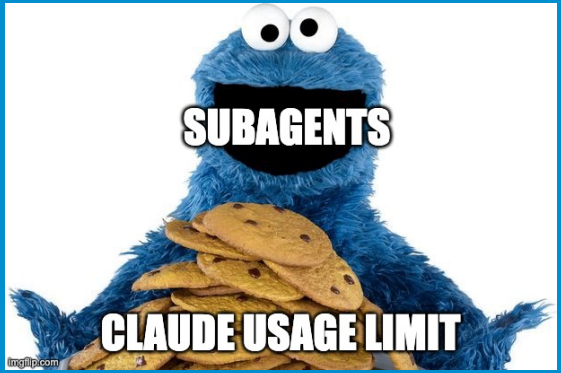
Example: `/summarize-paper` reads a paper end-to-end and returns a structured extract

5. Subagents turn one assistant into a team

The agent can spawn **many subagents** that work in parallel, each on a slice, then report back.

- **Reading list:** `/summarize-paper` fans out to multiple papers in parallel; done in the time it takes to read one
- **Paper audit:** `/referee2` runs five specialists simultaneously: code, replication, econometrics, structure, prose
- **LLM Council:** `/council` sends a question to four models; they critique each other and a chair synthesises

🎓 **Example:** 30 papers on peer effects summarized and synthesized in a coffee break



ME WANT TOKENS.

NOM NOM NOM.

30 subagents = 30× your quota.

Saves time, not tokens.

6. MCP connects the agent to other tools (like Zotero)

(**M**odel **C**ontext **P**rotocol) is a standard plug for external tools. Each has to be installed once.

With the Zotero MCP, I can:

- Ask:"What papers do I have on X?"
- Let Claude get the PDF directly from Zotero (incl. my highlights/notes)

Caution

- An MCP runs code on your machine. Install only what you trust.

Live demo: Let's try cleaning up some unstructured Zotero library

7. Hooks make the agent enforce your conventions

A hook is a small script the tool runs automatically at set moments: *before* a command, *after* an edit, at session start. Your rules, enforced by code, not by trust.

```
# Before any Bash command: block ‘rm‘, suggest archiving
guard-rm.sh        -> "use: mv <file> archive/"

# At session start: read log/current.md and brief the agent
session-start.sh   -> summarise what was in progress
```

🎓 **Example:** the agent opens with a one-line briefing on where the last session ended, without me typing anything.

8. Verification is the new bottleneck

"Verification is the new bottleneck. Production used to be the bottleneck of research, now verification is." — Scott Cunningham

The problem

- ⚠ Code, analyses, even papers are now almost costless to produce
- ⚠ The agent is confident even when it is wrong; it rarely says "I'm not sure"

Agents can help here too

- ✓ Write verification scripts, re-runnable, kept with the project
- ✓ Let the agent check its output against a known truth (e.g. with MC simulations)

🎓 **Example:** I had to slightly adapt a function from an R package. Claude wrote verification scripts to make sure the estimator still recovers the true value

The reveal + a note

Nothing is lost: every session is on your disk

The whole run is preserved as structured text. The work is **reproducible and auditable**.

```
~/.claude/projects/<working-dir>/<session-id>.jsonl
```

Inside: your original prompt (typos and all), every tool call, every subagent, every R script, every error and retry. A "flight recorder" for the work.

Why it matters: the quiet fear is that AI work is a black box. It is not. The full record sits on your machine, ready to send to a co-author or point another agent at.

Thank you.

Your turn.

Where has AI made you 10× faster?

Where has AI failed, subtly or spectacularly?